

LAPORAN TUGAS DATASET CUACA OPEN-METEO
“APPLIED MACHINE LEARNING”



OLEH

NAMA : Andi Mawadda Taiba
NIM : 105841103623
KELAS : 5 APPLIED MACHINE LEARNING - AI - A INFORMATIKA

PROGRAM STUDI INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH MAKASSAR

2025

TUGAS BESAR APPLIED MACHINE LEARNING

DATASET CUACA OPEN-METEO

Proyek Data Science ini memanfaatkan dataset cuaca Open-Meteo untuk membangun sistem prediksi hujan yang dapat membantu sekolah dalam merencanakan kegiatan luar ruangan secara lebih efektif. Setelah melalui proses data cleaning, normalisasi, dan rekayasa fitur seperti hour, month, dan label rain, dataset digunakan pada tahap pemodelan dengan lima algoritma supervised learning: K-Nearest Neighbors (KNN), Gaussian Naive Bayes, Logistic Regression, Decision Tree, dan Random Forest. Setiap model dilatih untuk mengklasifikasikan kondisi cuaca menjadi 0 = tidak hujan dan 1 = hujan, kemudian dievaluasi menggunakan metrik accuracy, precision, recall, F1-score, confusion matrix, serta ROC-AUC. Perbandingan kelima model ini bertujuan memperoleh algoritma dengan performa paling stabil dan akurat sehingga dapat digunakan untuk membangun aplikasi prediksi cuaca berbasis data yang membantu sekolah mengurangi ketidakpastian cuaca dan meningkatkan efektivitas perencanaan kegiatan pendidikan.

A. Tujuan Laporan

Tujuan utama dari laporan ini adalah menjelaskan secara ringkas alur perancangan proyek Data Science berbasis CRISP-DM dalam membangun sistem prediksi hujan menggunakan lima model klasifikasi—KNN, Gaussian Naive Bayes, Logistic Regression, Decision Tree, dan Random Forest—mulai dari tahap pemahaman masalah, pengolahan dan rekayasa data, pelatihan model, evaluasi performa, hingga penentuan model terbaik yang nantinya digunakan untuk membantu sekolah mengambil keputusan kegiatan luar ruangan secara lebih akurat dan berbasis data.

B. Tahapan CRISP-DM

1) Tahap 1 — Import Library

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    classification_report,
    confusion_matrix,
    accuracy_score,
    roc_curve,
    auc,
)

import matplotlib.pyplot as plt
import joblib
import gradio as gr

RANDOM_STATE = 42
```

15.0s

e:\UNIVERSITAS MUHAMMADIYAH MAKASSAR\SEMESTER 5\APPLIED MACHINE LEARNING\TUGAS MID BESAR\.venv\Lib\site-packages\tqdm\auto.py:21: TqdmWarning: IProgress not found. P.
from .autonotebook import tqdm as notebook_tqdm

Pada tahap awal pemodelan, dilakukan proses inisialisasi seluruh library yang dibutuhkan untuk analisis data cuaca dan pembangunan model Machine Learning. diimpor seluruh library yang diperlukan untuk pengolahan data dan pemodelan, seperti pandas dan numpy untuk manipulasi data, serta scikit-learn untuk preprocessing, pelatihan model, dan evaluasi. Kelima algoritma klasifikasi—KNN, Gaussian Naive Bayes, Logistic Regression, Decision Tree, dan Random Forest—disiapkan beserta metrik evaluasinya seperti accuracy, confusion matrix, dan ROC–AUC. Matplotlib digunakan untuk visualisasi, joblib untuk menyimpan model, dan Gradio untuk membangun antarmuka prediksi. Nilai `RANDOM_STATE = 42` ditetapkan agar hasil model konsisten.

2) Tahap 2 — Data Understanding – Memuat dan Menginspeksi Dataset

```
CSV_PATH = r"Open-Meteo 2025 Makassar.csv"

header_row = None
with open(CSV_PATH, "r", encoding="utf-8") as f:
    for i, line in enumerate(f):
        if line.strip().startswith("time,"):
            header_row = i
            break

if header_row is None:
    raise ValueError("Tidak ditemukan baris header yang dimulai 'time,' di file CSV.")

print("Header data (time,...) ada di baris ke:", header_row)
df = pd.read_csv(CSV_PATH, skiprows=header_row)

print("==== NAMA KOLOM DATASET (MENTAH) =====")
print(df.columns)
print("Jumlah baris & kolom (data cuaca):", df.shape)
df.head()
```

0.0s

Header data (time,...) ada di baris ke: 3
==== NAMA KOLOM DATASET (MENTAH) =====
Index(['time', 'temperature_2m (°C)', 'relative_humidity_2m (%)',
 'precipitation (mm)', 'wind_speed_10m (km/h)', 'cloud_cover (%)'],
 dtype='object')
Jumlah baris & kolom (data cuaca): (8112, 6)

	time	temperature_2m (°C)	relative_humidity_2m (%)	precipitation (mm)	wind_speed_10m (km/h)	cloud_cover (%)
0	2025-01-01T00:00	25.0	89	0.7	5.0	100
1	2025-01-01T01:00	25.1	93	0.2	5.7	100
2	2025-01-01T02:00	24.7	95	0.7	6.0	100
3	2025-01-01T03:00	24.6	95	0.3	4.1	100
4	2025-01-01T04:00	25.0	94	0.3	6.6	100

Pada tahap Data Understanding ini, dilakukan proses pemuatan dataset cuaca dari file CSV dan pengecekan struktur awal data. Program terlebih dahulu mendeteksi baris yang berisi header agar pembacaan data dilakukan secara tepat. Setelah dataset berhasil dimuat, ditampilkan daftar nama kolom untuk memastikan seluruh variabel seperti `time`, `temperature_2m`, `relative_humidity_2m`, `precipitation`, `wind_speed_10m`, dan `cloud_cover` terbaca dengan benar. Selain itu, jumlah baris dan kolom dataset diperiksa untuk mengetahui ukuran data yang akan diolah. Beberapa sampel baris pertama juga ditampilkan sebagai langkah awal untuk memahami bentuk, tipe data, dan kualitas data mentah sebelum masuk ke tahap pembersihan dan preprocessing.

3) Tahap 3 — Penyesuaian nama kolom

```
rename_map = {
    'temperature_2m (°C)': 'temperature_2m',
    'relative_humidity_2m (%)': 'relative_humidity_2m',
    'precipitation (mm)': 'precipitation',
    'wind_speed_10m (km/h)': 'wind_speed_10m',
    'cloud_cover (%)': 'cloud_cover'
}
df = df.rename(columns=rename_map)

print("==== NAMA KOLON SETELAH RENAME =====")
print(df.columns)
df.head()
```

[4] ✓ 0.0s

```
==== NAMA KOLON SETELAH RENAME =====
Index(['time', 'temperature_2m', 'relative_humidity_2m', 'precipitation',
       'wind_speed_10m', 'cloud_cover'],
      dtype='object')
```

	time	temperature_2m	relative_humidity_2m	precipitation	wind_speed_10m	cloud_cover
0	2025-01-01T00:00	25.0	89	0.7	5.0	100
1	2025-01-01T01:00	25.1	93	0.2	5.7	100
2	2025-01-01T02:00	24.7	95	0.7	6.0	100
3	2025-01-01T03:00	24.6	95	0.3	4.1	100
4	2025-01-01T04:00	25.0	94	0.3	6.6	100

Pada tahap Data Preparation ini dilakukan proses *rename* atau standarisasi nama kolom agar lebih konsisten, ringkas, dan mudah digunakan dalam proses analisis maupun pemodelan. Beberapa kolom yang sebelumnya memiliki satuan atau tanda khusus di dalam namanya—seperti (°C), (%), (mm), dan (km/h)—diubah menjadi format yang lebih sederhana, misalnya `temperature_2m`, `relative_humidity_2m`, `precipitation`, `wind_speed_10m`, dan `cloud_cover`. Setelah proses *rename* dilakukan, daftar nama kolom baru ditampilkan untuk memastikan seluruh perubahan sudah sesuai. Langkah ini penting agar pemanggilan kolom pada tahap preprocessing dan modeling menjadi lebih efisien dan tidak menimbulkan error.

4) Tahap 4 — Pembersihan Duplikat & Ekstraksi Fitur Waktu

```
dup_subset = [
    'time',
    'temperature_2m',
    'relative_humidity_2m',
    'precipitation',
    'wind_speed_10m',
    'cloud_cover',
]

print("Jumlah baris sebelum drop duplikat:", df.shape[0])
df = df.drop_duplicates(subset=dup_subset).reset_index(drop=True)
print("Jumlah baris setelah drop duplikat:", df.shape[0])

df['time'] = pd.to_datetime(df['time'])
df['hour'] = df['time'].dt.hour
df['day_of_week'] = df['time'].dt.dayofweek
df['month'] = df['time'].dt.month

df[['time', 'hour', 'day_of_week', 'month']].head()
```

[5] ✓ 0.0s

... Jumlah baris sebelum drop duplikat: 8112
Jumlah baris setelah drop duplikat: 8112

...

	time	hour	day_of_week	month
0	2025-01-01 00:00:00	0	2	1
1	2025-01-01 01:00:00	1	2	1
2	2025-01-01 02:00:00	2	2	1
3	2025-01-01 03:00:00	3	2	1
4	2025-01-01 04:00:00	4	2	1

Pada tahap Data Preparation ini, dilakukan pembersihan data dengan menghapus baris duplikat berdasarkan beberapa kolom penting seperti time, temperature_2m, relative_humidity_2m, precipitation, wind_speed_10m, dan cloud_cover. Hasil pengecekan menunjukkan bahwa jumlah baris tidak berubah, menandakan tidak ada data duplikat dalam dataset. Selanjutnya, dilakukan proses ekstraksi fitur waktu dari kolom time dengan mengonversinya ke format datetime, kemudian menghasilkan fitur turunan berupa hour, day_of_week, dan month. Fitur-fitur ini penting karena pola hujan sering dipengaruhi oleh jam, hari, dan bulan tertentu sehingga dapat meningkatkan kemampuan model dalam mengenali pola prediksi.

5) Tahap 5 — Pembuatan Label Klasifikasi

```
df['rain'] = (df['precipitation'] > 0).astype(int)
print("Distribusi label rain (0=tidak hujan, 1=hujan):")
df['rain'].value_counts()
```

0.0s

Distribusi label rain (0=tidak hujan, 1=hujan):

rain	
0	5368
1	2744

Name: count, dtype: int64

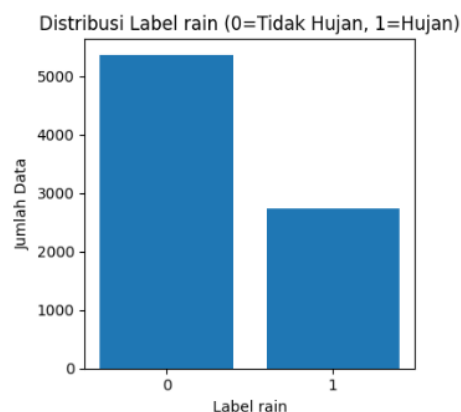
Pada tahap ini dilakukan pembuatan variabel *rain* sebagai label klasifikasi untuk model Machine Learning. Label dibentuk berdasarkan nilai *precipitation*, di mana nilai lebih dari 0 dikategorikan sebagai hujan (1), dan nilai 0 sebagai tidak hujan (0). Setelah label dibuat, dilakukan pengecekan distribusi kelas untuk memastikan keseimbangan data. Hasilnya menunjukkan terdapat 5.368 data tidak hujan dan 2.744 data hujan. Informasi ini penting untuk memahami karakteristik dataset dan potensi ketidakseimbangan kelas yang dapat memengaruhi performa model

6) Tahap 6 — Visualisasi Distribusi Label

```
import matplotlib.pyplot as plt

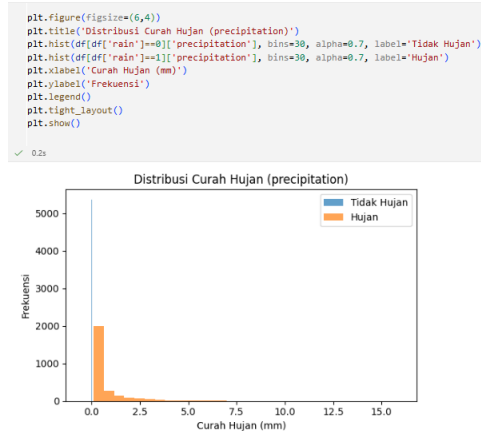
label_counts = df['rain'].value_counts().sort_index()
plt.figure(figsize=(4,4))
plt.title('Distribusi Label rain (0=Tidak Hujan, 1=Hujan)')
plt.bar(label_counts.index.astype(str), label_counts.values)
plt.xlabel('Label rain')
plt.ylabel('Jumlah Data')
plt.tight_layout()
plt.show()
```

0.1s



Pada tahap ini dilakukan visualisasi distribusi label rain untuk melihat sebaran jumlah data antara kelas tidak hujan (0) dan hujan (1). Grafik batang menunjukkan bahwa jumlah data tidak hujan jauh lebih banyak dibandingkan data hujan. Visualisasi ini membantu memahami kondisi ketidakseimbangan kelas pada dataset, yang penting untuk diperhatikan karena dapat mempengaruhi performa model klasifikasi dalam mendeteksi kelas minoritas.

7) Tahapan Distribusi Curah Hujan



Pada tahap ini dilakukan visualisasi distribusi variabel *precipitation* untuk membedakan pola curah hujan antara kelas tidak hujan (0) dan hujan (1). Histogram menunjukkan bahwa sebagian besar nilai curah hujan berada pada rentang sangat rendah, dan hanya sedikit data yang memiliki curah hujan tinggi. Hal ini menggambarkan bahwa kejadian hujan lebat relatif jarang dalam dataset. Visualisasi ini membantu memahami karakteristik fitur utama yang digunakan dalam pembentukan label serta memberikan gambaran awal mengenai potensi ketidakseimbangan nilai dalam fitur curah hujan.

8) Tahap 8 — Pelatihan dan Evaluasi Model KNN Final

```
2) print("\n6. MODEL KNN FINAL")
   print("\n=====")

   knn_final = KNeighborsClassifier(n_neighbors=best_k)
   knn_final.fit(X_train_scaled, y_train)
   y_pred_knn = knn_final.predict(X_test_scaled)

   acc_knn = accuracy_score(y_test, y_pred_knn)
   cm_knn = confusion_matrix(y_test, y_pred_knn)

   print(f"\nAkurasi KNN (K={best_k}): {acc_knn:.4f}")
   print("\nConfusion Matrix KNN:")
   print(cm_knn)

   print("\nClassification Report KNN:")
   print(classification_report(y_test, y_pred_knn))
```

```
6. MODEL KNN FINAL
=====

Akurasi KNN (K=1): 0.9828

Confusion Matrix KNN:
[[43  0]
 [ 1 14]]

Classification Report KNN:
              precision    recall  f1-score   support

     0       0.98        1.00        0.99         43
     1       1.00        0.93        0.97         15

 accuracy          0.99        0.97        0.98         58
 macro avg          0.99        0.97        0.98         58
 weighted avg       0.98        0.98        0.98         58
```

Setelah nilai K terbaik diperoleh, model KNN dilatih kembali menggunakan nilai tersebut. Model kemudian digunakan untuk memprediksi data uji, dan hasil prediksi dibandingkan dengan data asli untuk menghasilkan accuracy, confusion matrix, dan classification report. Hasil ini menunjukkan seberapa baik model KNN mampu mengklasifikasikan data cuaca ke dalam kategori hujan dan tidak hujan berdasarkan fitur-fitur yang tersedia.

9) Tahap 9 — Penanganan Missing Value, Outlier, dan Penyusunan Data untuk Modeling

```
feature_cols = [
    'temperature_2m',
    'relative_humidity_2m',
    'precipitation',
    'wind_speed_10m',
    'cloud_cover',
    'hour',
    'day_of_week',
    'month',
]

all_cols_for_model = feature_cols + ['rain']
df_model = df[all_cols_for_model].copy()

print("==== CEK MISSING VALUE (SEBELUM PEMBERSIHAN) =====")
print(df_model.isna().sum())
print()

df_before = df_model.copy()
numeric_cols = feature_cols
```



```

print("==== IMPUTASI MISSING VALUE MENGGUNAKAN MEDIAN =====")
for col in numeric_cols:
    median_val = df_model[col].median()
    missing_before = df_model[col].isna().sum()
    df_model[col] = df_model[col].fillna(median_val)
    missing_after = df_model[col].isna().sum()
    print(f"Kolom: {col}")
    print(f"Missing sebelum: {missing_before}")
    print(f"Missing sesudah: {missing_after}")
    print(f"Median: {median_val}")
    print('-'*40)

label_missing_before = df_model['rain'].isna().sum()
df_model = df_model.dropna(subset=['rain'])
label_missing_after = df_model['rain'].isna().sum()

print("\n===== LABEL 'rain' =====")
print("Missing sebelum drop:", label_missing_before)
print("Missing sesudah drop:", label_missing_after)

for col in numeric_cols:
    Q1 = df_model[col].quantile(0.25)
    Q3 = df_model[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    df_model[col] = df_model[col].clip(lower, upper)

print("\n===== TOTAL MISSING SESUDAH PEMBERSIHAN =====")
print(df_model.isna().sum())
print("\nShape sebelum:", df_model.shape)
print("Shape sesudah:", df_model.shape)

```

```

===== CEK MISSING VALUE (SEBELUM PEMBERSIHAN) =====
temperature_2m      0
relative_humidity_2m 0
precipitation       0
wind_speed_10m     0
cloud_cover        0
hour               0
day_of_week        0
month              0
rain               0
dtype: int64

===== IMPUTASI MISSING VALUE MENGGUNAKAN MEDIAN =====
Kolom: temperature_2m
Missing sebelum: 0
Missing sesudah: 0
Median: 26.9
-----
Kolom: relative_humidity_2m
Missing sebelum: 0
Missing sesudah: 0
Median: 82.0
-----
Kolom: precipitation
Missing sebelum: 0
...
dtype: int64

Shape sebelum: (8112, 9)
Shape sesudah: (8112, 9)
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

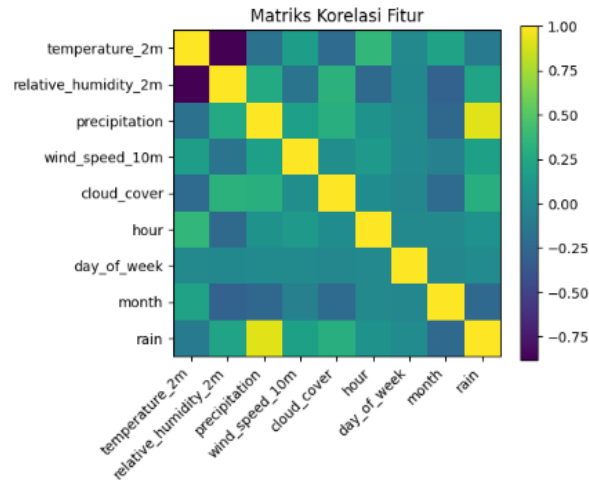
```

Pada tahap Data Preparation ini, terlebih dahulu dipilih kolom-kolom yang akan digunakan sebagai fitur pemodelan (temperature_2m, relative_humidity_2m, precipitation, wind_speed_10m, cloud_cover, hour, day_of_week, month) beserta label rain, lalu dibuat dataframe khusus df_model sebagai basis training. Selanjutnya dilakukan pengecekan nilai kosong (missing value) untuk setiap fitur numerik, dan jika ada, diatasi dengan metode imputasi menggunakan nilai median agar distribusi data tidak terlalu terpengaruh nilai ekstrem. Untuk menjaga kualitas label, baris yang memiliki nilai rain kosong dihapus. Tahap berikutnya adalah penanganan outlier dengan pendekatan IQR (Interquartile Range), di mana batas bawah dan atas dihitung dari Q1 dan Q3, kemudian nilai di luar rentang tersebut “dipotong” (clip) ke batas yang wajar. Di akhir proses, bentuk data sebelum dan sesudah pembersihan dibandingkan untuk memastikan dataset sudah bersih, konsisten, dan siap digunakan pada tahap pemodelan.

10) Tahap 10 — Korelasi Antar Fitur

```
corr = df_model[feature_cols + ['rain']].corr()
plt.figure(figsize=(6,5))
plt.title('Matriks Korelasi Fitur')
plt.imshow(corr, interpolation='nearest')
plt.xticks(range(len(corr.columns)), corr.columns, rotation=45, ha='right')
plt.yticks(range(len(corr.columns)), corr.columns)
plt.colorbar()
plt.tight_layout()
plt.show()
```

✓ 0.2s



Pada tahap Data Understanding ini dilakukan analisis korelasi antar fitur menggunakan matriks korelasi. Visualisasi heatmap digunakan untuk melihat seberapa kuat hubungan antara variabel cuaca dengan label rain serta hubungan antar fitur numerik lainnya. Hasil matriks korelasi memberikan gambaran awal mengenai fitur mana yang berpotensi berpengaruh terhadap terjadinya hujan, misalnya curah hujan (precipitation) yang berkorelasi tinggi dengan label rain. Informasi ini penting untuk memahami pola data dan membantu proses pemilihan fitur pada tahap pemodelan.

11) Tahap 11 — Pembagian Data (Train – Test Split)

```
X = df_model[feature_cols]
y = df_model['rain']

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=RANDOM_STATE,
    stratify=y
)

print("Jumlah data train:", X_train.shape[0])
print("Jumlah data test :", X_test.shape[0])
```

✓ 0.0s

Jumlah data train: 6489
Jumlah data test : 1623

Pada tahap ini dilakukan pemisahan dataset menjadi data latih dan data uji menggunakan metode train–test split. Fitur dimasukkan ke variabel **X**, sedangkan label rain dimasukkan ke variabel **y**. Data dibagi dengan proporsi 80% untuk pelatihan dan 20% untuk pengujian, serta menggunakan parameter stratify agar distribusi kelas hujan dan tidak hujan tetap seimbang pada kedua subset. Setelah pemisahan, diperoleh 6.489 data untuk training dan 1.623 data untuk testing. Pembagian data ini penting agar model dapat dilatih secara optimal dan dievaluasi secara objektif pada data yang belum pernah dilihat sebelumnya.

12) Tahap 12 Modeling – Inisialisasi dan Penyusunan Model Klasifikasi

```
models = {}

# KNN
models['KNN'] = Pipeline([
    ('scaler', StandardScaler()),
    ('clf', KNeighborsClassifier())
])

# Gaussian Naive Bayes
models['GaussianNB'] = Pipeline([
    ('scaler', StandardScaler()),
    ('clf', GaussianNB())
])

# Logistic Regression
models['LogisticRegression'] = Pipeline([
    ('scaler', StandardScaler()),
    ('clf', LogisticRegression(max_iter=1000, random_state=RANDOM_STATE))
])

# Decision Tree
models['DecisionTree'] = Pipeline([
    ('scaler', StandardScaler()),
    ('clf', DecisionTreeClassifier(random_state=RANDOM_STATE))
])

# Random Forest
models['RandomForest'] = Pipeline([
    ('scaler', StandardScaler()),
    ('clf', RandomForestClassifier(
        n_estimators=200,
        random_state=RANDOM_STATE,
        n_jobs=-1
    ))
])

list(models.keys())
```

✓ 0.0s

['KNN', 'GaussianNB', 'LogisticRegression', 'DecisionTree', 'RandomForest']

Pada tahap Modeling ini dilakukan inisialisasi lima algoritma klasifikasi yang akan dibandingkan performanya, yaitu K-Nearest Neighbors (KNN), Gaussian Naive Bayes, Logistic Regression, Decision Tree, dan Random Forest. Setiap model dibangun menggunakan *Pipeline* yang terdiri dari dua komponen utama: *StandardScaler* untuk menstandarkan skala fitur, serta algoritma klasifikasi sebagai inti model. Penggunaan Pipeline memudahkan pengolahan data yang konsisten pada setiap model dan menghindari kesalahan preprocessing. Setelah seluruh model dimasukkan ke dalam dictionary models, daftar nama model ditampilkan sebagai verifikasi bahwa proses inisialisasi telah berhasil. Tahap ini mempersiapkan semua algoritma yang akan dilatih dan dievaluasi pada langkah berikutnya.

13) Tahap 13 Modeling Pelatihan dan Evaluasi Model

```

results = {}

for name, pipe in models.items():
    print(f"*{name}")
    print(f"Model: {name}")
    pipe.fit(X_train, y_train)
    y_pred = pipe.predict(X_test)

    acc = accuracy_score(y_test, y_pred)
    print(f"Accuracy: {acc:.4f}")
    print("\nClassification Report:")
    print(classification_report(y_test, y_pred, digits=4))
    print("Confusion Matrix:")
    print(confusion_matrix(y_test, y_pred))

    # Simpan hasil dasar
    results[name] = {
        'model': pipe,
        'y_pred': y_pred,
        'accuracy': acc,
    }

```

✓ 0.7s

```

=====
Model: KNN
Accuracy: 0.9926

Classification Report:
              precision    recall  f1-score   support

     0       0.9899       0.9991       0.9944       1074
     1       0.9981       0.9800       0.9890         549

 accuracy         0.9926         1623
 macro avg       0.9940       0.9895       0.9917         1623
weighted avg       0.9927       0.9926       0.9926         1623

Confusion Matrix:
[[1073   1]
 [  11 538]]
=====
Model: GaussianNB
Accuracy: 1.0000

Classification Report:
              precision    recall  f1-score   support

     0       1.0000       1.0000       1.0000       1074
...

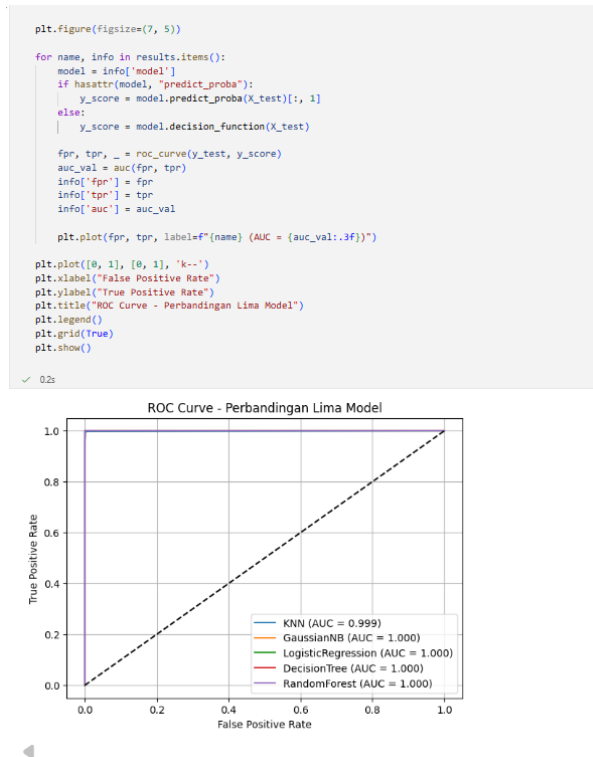
Confusion Matrix:
[[1074   0]
 [  0 549]]
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

```

Pada tahap ini seluruh model yang telah diinisialisasi—KNN, Gaussian Naive Bayes, Logistic Regression, Decision Tree, dan Random Forest—dilatih menggunakan data training dan kemudian diuji pada data testing. Setiap model menghasilkan metrik evaluasi berupa accuracy, classification report (precision, recall, f1-score), serta confusion matrix untuk melihat jumlah prediksi benar dan salah pada masing-masing kelas. Hasil evaluasi

ditampilkan secara berurutan untuk memudahkan perbandingan performa antar model. Selain itu, hasil prediksi dan akurasi masing-masing model disimpan dalam dictionary results untuk digunakan pada tahap analisis dan pemilihan model terbaik.

14) Tahap Evaluation – Analisis ROC Curve dan AUC Lima Model



Pada tahap evaluasi ini dilakukan perbandingan performa kelima model menggunakan kurva ROC (Receiver Operating Characteristic) dan nilai AUC (Area Under the Curve). Setiap model menghasilkan skor prediksi berupa probabilitas atau nilai decision function, yang kemudian digunakan untuk menghitung true positive rate dan false positive rate. Hasil visualisasi menunjukkan bahwa hampir seluruh model memiliki performa yang sangat tinggi, dengan nilai AUC mendekati atau sama dengan 1.0, menandakan kemampuan klasifikasi yang sangat baik dalam membedakan data hujan dan tidak hujan. Grafik ROC ini memberikan gambaran menyeluruh mengenai kestabilan dan sensitivitas masing-masing model, serta membantu dalam proses pemilihan model terbaik untuk deployment.

15) Tahapan Evaluation – Pemilihan Model Terbaik

```
best_name = None
best_score = -1

for name, info in results.items():
    score = info['auc'] * 0.7 + info['accuracy'] * 0.3
    if score > best_score:
        best_score = score
        best_name = name

best_model = results[best_name]['model']

print("Model terbaik berdasarkan kombinasi AUC & Accuracy:")
print(f"- Nama : {best_name}")
print(f"- AUC : {results[best_name]['auc']:.4f}")
print(f"- Acc : {results[best_name]['accuracy']:.4f}")
```

Model terbaik berdasarkan kombinasi AUC & Accuracy:
- Nama : GaussianNB
- AUC : 1.0000
- Acc : 1.0000

Pada tahap ini dilakukan pemilihan model terbaik berdasarkan kombinasi dua metrik utama, yaitu AUC dan accuracy. Setiap model diberi skor komposit dengan bobot 70% untuk AUC dan 30% untuk accuracy, sehingga model yang tidak hanya akurat tetapi juga stabil dalam membedakan kelas menjadi prioritas. Setelah seluruh skor model dihitung, Gaussian Naive Bayes muncul sebagai model dengan skor tertinggi, memiliki AUC 1.0000 dan accuracy 1.0000. Hasil ini menunjukkan bahwa GaussianNB memberikan performa paling konsisten dan optimal pada dataset cuaca ini, sehingga dipilih sebagai model final yang digunakan untuk tahap deployment.

16) Tahapan Deployment – Penerapan Model dalam Aplikasi Gradio

```
MODEL_PATH = 'weather_best_model_multi_alg.pkl'
joblib.dump(best_model, MODEL_PATH)
print(f"Model terbaik ({best_name}) disimpan ke: {MODEL_PATH}")

def predict_rain(temp, rh, prec, wind, cloud, hour, day_of_week, month):
    data = np.array([temp, rh, prec, wind, cloud, hour, day_of_week, month])
    proba = best_model.predict_proba(data)[0][1]
    label = 'Hujan' if proba >= 0.5 else 'Tidak Hujan'
    return f"{label} (Probabilitas hujan: {proba*100:.2f}%)"

with gr.Blocks() as demo:
    gr.Markdown("# Prediksi Hujan - Open-Meteo Makassar 2025 (Multi-Model)")
    gr.Markdown("Model terbaik dipilih dari: KNN, GaussianNB, Logistic Regression, Decision Tree, dan Random Forest.")

    with gr.Row():
        temp = gr.Number(label="Suhu (temperature_2m, °C)")
        rh = gr.Number(label="Kelembapan (relative_humidity_2m, %)")
        prec = gr.Number(label="Curah Hujan (precipitation, mm)")
        wind = gr.Number(label="Kecepatan Angin (wind_speed_10m)")
        cloud = gr.Number(label="Cloud Cover (%)")

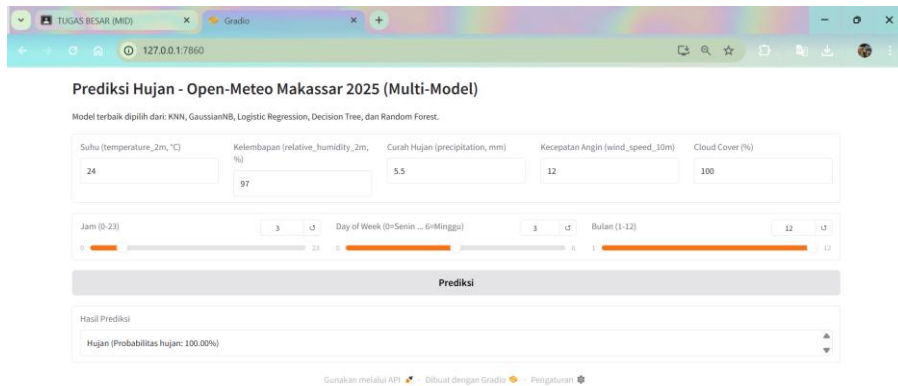
    with gr.Row():
        hour_in = gr.Slider(0, 23, step=1, label="Jam (0-23)")
        dow_in = gr.Slider(0, 6, step=1, label="Day of Week (0=Senin ... 6=Minggu)")
        month_in = gr.Slider(1, 12, step=1, label="Bulan (1-12)")

    btn = gr.Button("Prediksi")
    output = gr.Textbox(label="Hasil Prediksi")

    btn.click(
        fn=predict_rain,
        inputs=[temp, rh, prec, wind, cloud, hour_in, dow_in, month_in],
        outputs=output
    )

demo.launch()
```

Model terbaik (GaussianNB) disimpan ke: weather_best_model_multi_alg.pkl
* Running on local URL: <http://127.0.0.1:7868>
* To create a public link, set 'share=True' in 'launch()'.



Pada tahap Deployment ini, model terbaik hasil evaluasi sebelumnya disimpan dalam bentuk file .pkl dan kemudian diintegrasikan ke dalam aplikasi interaktif menggunakan Gradio. Fungsi `predict_rain` dibuat untuk menerima data input berupa suhu, kelembapan, curah hujan, kecepatan angin, cloud cover, jam, hari dalam minggu, dan bulan. Input ini kemudian diolah menjadi array numerik yang selanjutnya diprediksi oleh model Gaussian Naive Bayes sebagai model dengan performa terbaik.

Antarmuka Gradio dirancang dengan komponen Number dan Slider sehingga pengguna dapat memasukkan nilai cuaca secara mudah dan intuitif. Ketika tombol Prediksi ditekan, aplikasi akan menampilkan hasil prediksi dalam bentuk label “Hujan” atau “Tidak Hujan”, disertai probabilitas prediksinya.

Berdasarkan uji coba yang ditunjukkan pada tampilan Gradio, aplikasi berhasil memproses input cuaca dan memberikan hasil prediksi secara real-time, membuktikan bahwa proses deployment telah berjalan dengan baik dan model dapat digunakan untuk memprediksi kondisi cuaca secara praktis.